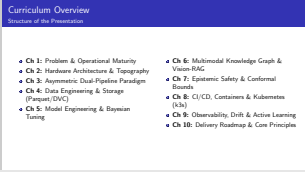
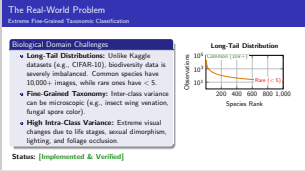


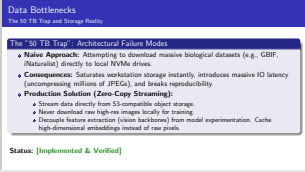
1. **Script:** Welcome everyone. Today we are walking through the TaxonVision-MLOps Strategy Report. This presentation is designed specifically for data scientists and engineers transitioning from introductory machine learning to production-grade MLOps. We will explore how real-world biodiversity classification forces us to rethink everything from storage and GPU time-slicing to mathematical safety bounds and continuous deployment.
2. **Q:** *What is the primary objective of this presentation?*
   A: To provide a comprehensive, end-to-end breakdown of the TaxonVision MLOps architecture, explaining why toy machine learning workflows fail at scale and how enterprise-grade systems guarantee reproducibility, latency, and epistemic safety.



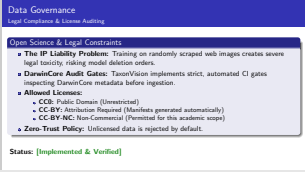
1. **Script:** Here is our roadmap for the presentation. We will progress systematically from real-world biodiversity problems and bare-metal hardware through data engineering, model optimization, epistemic uncertainty guarantees, and continuous Kubernetes deployment.
2. **Q:** *How is this curriculum organized?*
   A: It is structured into 10 modular chapters spanning 40+ slides, moving from introductory foundations to advanced hardware, mathematical calibration, and active learning.



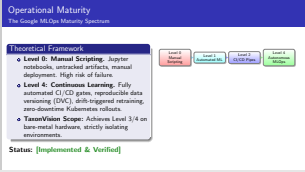
1. **Script:** When most students build computer vision models, they work with clean datasets like Dogs vs Cats or ImageNet. In real biodiversity monitoring, we deal with extreme long-tail distributions: a few common birds have 50,000 photos, while thousands of rare insects have two photos. Worse, two distinct species may differ only by microscopic vein patterns, while two photos of the exact same plant look completely different across seasons. We need a system that handles extreme fine-grained taxonomy without hallucinating confidence.
2. **Q:** *Why can't we simply collect more images for the rare species?*
   A: Biological rarity is governed by nature, not data collection budgets. An endangered lichen or beetle may only be sighted once every few years. The long-tail distribution is an immutable domain reality that our learning algorithms must solve mathematically.



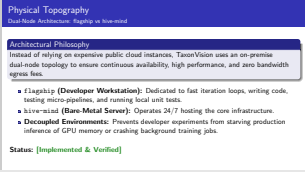
1. **Script:** Here is the most common mistake made by data scientists transitioning from bootcamps to industry: they run a script to download the entire dataset onto their local SSD. At 50 terabytes, your drive fills up in minutes, your network bandwidth is choked for days, and your coworker cannot reproduce your experiment without repeating the download. We call this the '50 TB Trap'. In TaxonVision, we enforce zero-copy streaming directly from object storage and cache immutable high-dimensional embeddings instead of raw pixels.
2. **Q:** *Does streaming images directly from S3 slow down model training?*
   A: If you train an end-to-end model on raw pixels from S3, network bandwidth becomes your bottleneck. That is why our architecture decouples feature extraction from classifier training: backbones process images in batch streams once, and classifiers train in seconds from pre-cached numerical embeddings.



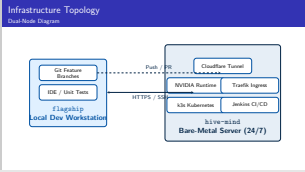
1. **Script:** Many engineers believe data pipelines are purely technical, but legal compliance is a hard engineering requirement. If your training set accidentally absorbs proprietary or copyrighted images, your entire model weights can become legally toxic and subject to deletion orders. We built an automated license audit script directly into our CI gate that inspects DarwinCore metadata and guarantees that only CC0, CC-BY, and CC-BY-NC images ever enter our pipelines.
2. **Q:** *What happens if an ingested image has no license field in its metadata?*
   A: The audit script rejects it by default under a strict zero-trust policy. Unlicensed data is treated as copyrighted until explicit open licensing is proven.



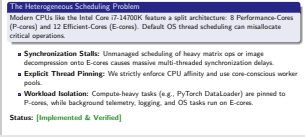
1. **Script:** To understand where TaxonVision sits, we look at the MLOps Maturity Model. Level 0 is what is taught in introductory courses: you run a Jupyter notebook, manually download a CSV, fit a model, and save a pickle file. If your data changes, you rerun the notebook by hand. Level 4 is an autonomous system: data ingestion triggers automated feature extraction, Bayesian optimization finds optimal weights, mathematical gates verify conformal coverage, and Kubernetes rolls out the updated container without downtime.
2. **Q:** *What is the biggest operational hurdle when moving from Level 0 to Level 4?*
   A: Automated validation gates. At Level 0, a human looks at an accuracy metric and decides to deploy. At Level 4, algorithms must autonomously decide if a candidate model is statistically safe and performant enough to replace the current production version.



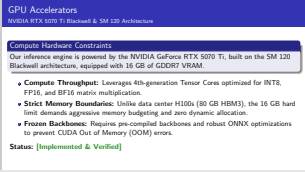
1. **Script:** In cloud-centric courses, instructors often assume you have an unlimited AWS budget. In reality, renting multiple A100 GPUs 24/7 costs thousands of dollars every month. We designed an on-premise dual-node topology: 'flagship' is our local development workstation where engineers develop with fast iteration loops, and 'hive-mind' is a 24/7 bare-metal server that hosts our k3s Kubernetes cluster, Jenkins automation, and production workloads.
2. **Q:** *Why not run everything on a single powerful machine?*
   A: Decoupling development from production prevents developer experiments from starving production inference of GPU memory or crashing background pipeline jobs.



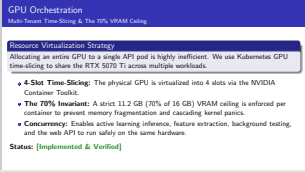
1. **Script:** This diagram visualizes our physical separation. The flagship node is strictly for local dev. Once code is pushed, Jenkins running inside the hive-mind k3s cluster takes over. The Cloudflare Tunnel securely routes external requests to our Traefik ingress without exposing any ports.
2. **Q:** *Is the network traffic between these nodes encrypted?*
   A: Yes, internal traffic can be routed over local LAN, while external web hooks and operator access run through the Cloudflare Zero Trust encrypted tunnel.



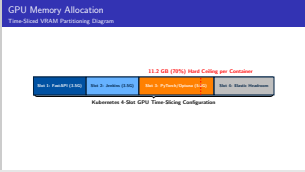
1. **Script:** Modern CPUs are no longer uniform; the i7-14700K combines high-frequency P-cores with low-power E-cores. If the Linux OS scheduler accidentally schedules an image decompression worker on an E-core while PyTorch is waiting on a P-core, your entire batch pipeline crawls to a halt. We use CPU affinity and core-conscious worker pools to ensure that compute-heavy tasks stay on P-cores while telemetry and background logging run on E-cores.
2. **Q:** *Does standard PyTorch DataLoader handle P-core vs E-core allocation automatically?*
   A: No. The standard OS scheduler treats all logical threads equally unless explicit taskset affinity or cgroup constraints are configured at the container runtime level.



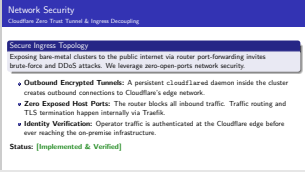
1. **Script:** Our bare-metal compute engine is powered by an NVIDIA RTX 5070 Ti based on the cutting-edge Blackwell SM 120 architecture. While it delivers extraordinary compute throughput with 4th-generation Tensor Cores and GDDR7 bandwidth, it has exactly 16 gigabytes of VRAM. If your model allocates 16.1 gigabytes, your job crashes instantly with a CUDA Out of Memory error. This physical constraint dictates our entire architectural design.
2. **Q:** *Why not use distributed multi-GPU training with cloud instances?*
   A: A single modern GPU on bare metal is more than capable of handling enterprise-scale classification if you engineer your pipelines correctly. Bounding ourselves to 16 GB forces disciplined, efficient engineering rather than brute-force cloud spending.



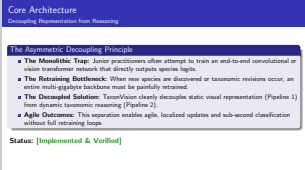
1. **Script:** In enterprise Kubernetes, allocating an entire physical GPU to a single API pod is massively wasteful because inference only uses the GPU intermittently. We implemented 4-slot GPU time-slicing, allowing our cluster to share the RTX 5070 Ti across multiple microservices. To ensure that one runaway job doesn't crash the entire GPU, we enforce an ironclad 70 percent VRAM cap—11.2 gigabytes—giving us a safety margin against memory fragmentation.
2. **Q:** *Does GPU time-slicing isolate memory between containers?*
   A: Time-slicing shares compute cycles in temporal slices, but does not provide hardware-enforced memory partitioning like MIG (Multi-Instance GPU). That is exactly why our 70% software cap and zero-allocation inference invariants are critical to prevent crashes.



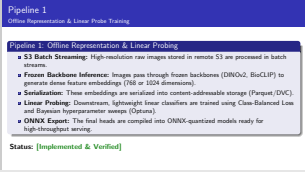
1. **Script:** This diagram shows the 16 GB VRAM physical capacity broken down into our 4 time-sliced slots. Slot 1 runs our FastAPI serving at up to 3.5 GB. Slot 2 is our Jenkins CI agent. Slot 3 is gated for PyTorch training at 5 GB. We deliberately leave Slot 4 as elastic headroom. The red dashed line illustrates our 70% container software ceiling, ensuring no single pod can monopolize the card and cause OOM cascading failures.
2. **Q:** *How is the 5.0 GB limit for PyTorch enforced?*
   A: We use `torch.cuda.set_per_process_memory_fraction(0.3125)` to explicitly restrict PyTorch to exactly 5.0 GB of the 16.0 GB total, guaranteeing it respects its designated slot bounds.



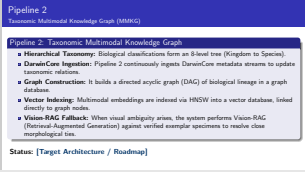
1. **Script:** How do we safely access our on-premise cluster from anywhere in the world without exposing it to internet hackers? We do not open any ports on our router. Instead, a lightweight daemon called cloudflared establishes an outbound encrypted tunnel to Cloudflare's global edge network. When an operator connects, Cloudflare authenticates their identity and proxies the traffic securely into our internal Kubernetes ingress controller.
2. **Q:** *What happens if the local internet connection resets its dynamic IP address?*
   A: The tunnel daemon maintains persistent outbound connections. If the local public IP changes, the daemon automatically reconnects without any downtime or manual DNS reconfiguration.



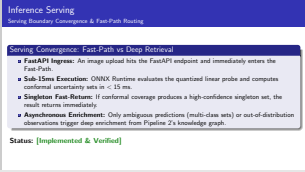
1. **Script:** This slide represents the core architectural breakthrough of our system: the Asymmetric Dual-Pipeline. In naïve computer vision, you take an image, pass it through ResNet, and output 1,000 class probabilities. But in biology, taxonomy changes every month: species are split, renamed, or reclassified. If you use a monolithic model, every single rename forces a full 10-hour retraining run on expensive GPUs. By decoupling representation from reasoning, our foundation model stays entirely frozen while our taxonomic knowledge graph updates in real time with zero mathematical friction.
2. **Q:** *What is a foundation model?*
   A: A vision transformer (BioCLIP, DINOv2) pre-trained on biological imagery using self-supervised learning, yielding rich morphological representations without task-specific retraining.
3. **Q:** *Why "Asymmetric"?*
   A: Pipelines operate at different timescales: Pipeline 1 uses dense matrix ops on GPUs, while Pipeline 2 uses graph traversals and vector lookups on CPUs.



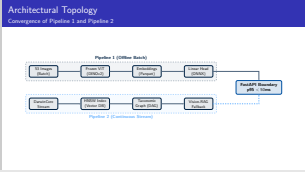
1. **Script:** Pipeline 1 is our offline training engine. Instead of feeding raw images directly into training loops every epoch, we pass them through our frozen backbone once and store the resulting numerical embeddings on disk. Once those embeddings are saved, training a new classification head takes literally ten seconds. We can run 50 hyperparameter optimization trials with Optuna in five minutes on a single GPU—something that would take days with full end-to-end fine-tuning. This is how we achieve extreme ML operational efficiency.
2. **Q:** *Does training a linear head hurt accuracy?*
   A: Self-supervised backbones produce features where linear probes achieve within 1–2% of full fine-tuning, while saving orders of magnitude in compute and preventing catastrophic forgetting.
3. **Q:** *What is Class-Balanced Loss?*
   A: A loss weighting technique correcting for long-tail biological distributions, ensuring rare species are not ignored by the model.



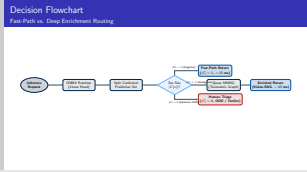
1. **Script:** While Pipeline 1 provides raw computational classification, Pipeline 2 provides actual biological wisdom. Taxonomy is not a flat list of labels; it is an evolving phylogenetic tree. Pipeline 2 stores this tree in a Multimodal Knowledge Graph. If the classifier is mathematically uncertain whether a specimen is Species A or Species B, Pipeline 2 queries their taxonomic relations, habitat metadata, and anatomical descriptions—acting exactly like a human field botanist cross-referencing herbarium records in real-time.
2. **Q:** *How does the vector database fit into the knowledge graph?*
   A: Each taxonomic node is linked to reference exemplar embeddings in our vector store. This allows us to perform nearest-neighbor visual similarity searches constrained entirely within specific sub-branches of the taxonomic tree, vastly reducing search space.
3. **Q:** *What does DarwinCore ingestion provide?*
   A: DarwinCore is the global standard for biodiversity data. It provides the standardized metadata—such as geographic coordinates, collector information, and accepted scientific names—required to construct the taxonomic graph accurately.



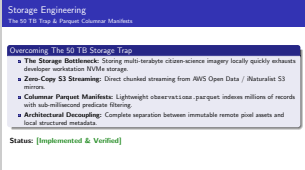
1. **Script:** In production APIs, speed is everything. We simply cannot afford to query a complex graph database on every single simple request. We implement a Fast-Path serving boundary: the incoming image runs through our ultra-fast ONNX model. If the prediction is unambiguous—meaning the conformal predictor returns exactly one class—the API responds in under 15 milliseconds. Only when the model mathematically detects ambiguity does it trigger the deep, slightly slower, taxonomic enrichment pipeline.
2. **Q:** *What is ONNX Runtime and why do we use it instead of native PyTorch?*
   A: ONNX Runtime optimizes the computation graph, fuses operations, and performs strict memory reuse. It eliminates Python interpreter overhead, resulting in 2x to 4x lower latency and rigorously enforces zero memory allocations on the hot inference path.
3. **Q:** *What is conformal uncertainty?*
   A: It is a mathematically rigorous framework that outputs a set of possible classes rather than a single point prediction, guaranteeing with a user-specified probability (e.g., 95%) that the true class is contained within that set.



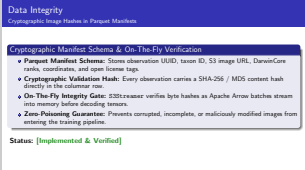
1. **Script:** This diagram visualizes our asymmetric dual-pipeline. The top row is Pipeline 1: heavy, offline batch processing where raw images become dense embeddings, then fast ONNX models. The bottom row is Pipeline 2: lightweight, continuous DarwinCore ingestion that builds our multimodal knowledge graph. Notice how they only meet at the FastAPI serving boundary on the right. This topological isolation is what prevents cascading retraining failures.
2. **Q:** *Why does the arrow from Vision-RAG to FastAPI use a dashed line?*
   A: Because Vision-RAG is our fallback mechanism. It is not triggered on every request. It only executes asynchronously when the Fast-Path linear head detects conformal uncertainty.



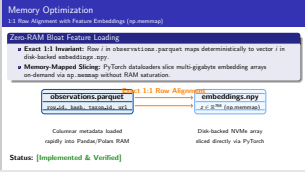
1. **Script:** Here we zoom into the FastAPI convergence boundary. Every request runs through ONNX and the conformal predictor. If the conformal set size is exactly one, it means the model is mathematically confident, and we immediately return via the Fast-Path in under 15 milliseconds. If the set size is greater than one, it means the prediction is ambiguous, and we route the request to the deeper Taxonomic Graph for Vision-RAG enrichment. If the set is empty, it routes to human triage.
2. **Q:** *What happens if the conformal set is empty (Size = 0)?*
   A: An empty set implies the image is an epistemic or out-of-distribution outlier. In this case, the system safely routes the observation to the active learning human review queue and returns an unclassified referral flag.



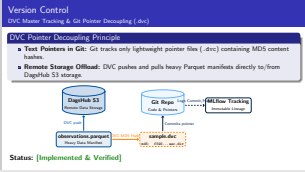
1. **Script:** In large-scale biological vision, downloading 50 terabytes of raw images is impossible for individual developers. Instead of downloading JPEGs, TaxonVision uses lightweight Apache Parquet index manifests. The images remain on remote AWS Open Data S3 buckets, while our local system works with columnar metadata that can be filtered and sliced in milliseconds.
2. **Q:** *Why use Apache Parquet instead of an SQLite or PostgreSQL database?*
   A: Parquet provides columnar compression, partitioned scanning, and native zero-copy integration with Apache Arrow, PyTorch, and Polars, making it ideal for distributed MLOps pipelines without database server overhead.



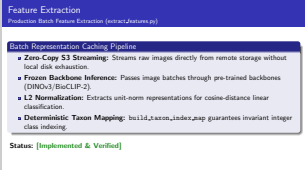
1. **Script:** How do we ensure that an image on S3 hasn't been corrupted or altered over time? Inside our Parquet manifest, every single observation has a cryptographic SHA-256 validation hash stored alongside its S3 URL. When our S3Streamer module streams the image into memory, it checks the SHA-256 hash immediately before decoding. If a single byte is corrupted, the observation is caught and logged instantly.
2. **Q:** *Does on-the-fly hash verification introduce CPU bottlenecks?*
   A: SHA-256 hashing in hardware-accelerated CPU instructions takes microseconds per image buffer, which is entirely hidden behind network transfer and image decoding latency.



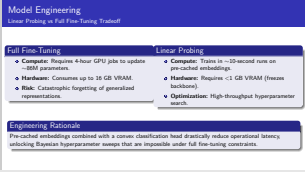
1. **Script:** Once features are extracted, how do we load them without eating 30 gigabytes of RAM? We enforce an exact 1:1 row alignment between our Parquet manifest and a disk-backed NumPy memmap file. Row 500 in Parquet corresponds precisely to vector 500 in embeddings.npy. PyTorch dataloaders can slice any batch directly from NVMe storage with zero RAM allocation overhead.
2. **Q:** *What happens if an observation is deleted or filtered out?*
   A: The entire split is re-indexed deterministically during feature extraction, guaranteeing that Parquet rows and memmap indices never fall out of synchronization.



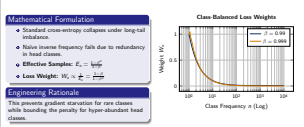
1. **Script:** At the version control level, we never put Parquet files or embeddings into Git. DVC computes a master MD5 hash of the entire Parquet manifest and saves it in a tiny text file like data/sample.dvc. Git commits the text file, and DVC syncs the heavy Parquet file to our DagsHub S3 remote. When a developer switches Git branches, DVC automatically pulls the exact matching manifest.
2. **Q:** *Can MLflow link training runs to the DVC commit hash?*
   A: Yes. Our training runner logs the active Git commit hash and DVC data hash directly into MLflow run metadata, providing an unbreakable audit trail from model weights back to exact dataset rows.



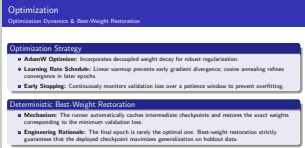
1. **Script:** In `extract_features.py`, we execute our batch feature extraction. It connects to S3, loads batches of images through our frozen vision backbone, normalizes the embeddings, and writes them out into compact columnar Parquet files. Crucially, it builds a global taxon index map that deterministically assigns each species an integer index across all train, validation, and test splits. This guarantees that class 42 is always identical across all experiments.
2. **Q:** *Why use Parquet format for storing numerical feature embeddings?*
   A: Parquet is a compressed columnar format with zero-copy deserialization in Python. Loading a 1 GB Parquet file into memory takes milliseconds compared to slow JSON or CSV parsing.



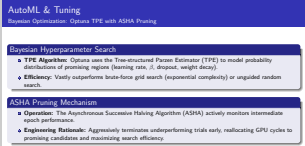
1. **Script:** Let's compare the economics of linear probing versus full fine-tuning. Full fine-tuning requires backpropagating through 86 million transformer parameters, consuming 14 gigabytes of VRAM and 4 hours of GPU time per run. Linear probing trains a single linear projection matrix on pre-extracted embeddings. It uses 500 megabytes of VRAM and finishes in 8 seconds. This 1,000x speedup allows us to explore hyperparameter spaces thoroughly rather than settling for un-tuned defaults.
2. **Q:** *Can linear probing adapt if the test domain has different lighting or camera sensors?*
   A: Because the foundation model was pre-trained on diverse web-scale imagery, its internal representations already generalize well across lighting and sensor variations without needing backbone parameter updates.



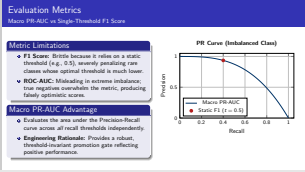
1. **Script:** Standard cross-entropy loss fails on long-tail data because the gradients from 10,000 common robins overwhelm the gradients from 3 rare warblers. But if you simply weight by inverse frequency, you over-weight outliers and make training unstable. We use Class-Balanced Loss, which calculates the 'effective number of samples' using a hyperparameter beta. It mathematically accounts for information redundancy in large classes, providing stable, well-balanced gradients.
2. **Q:** *What is the typical value of β in our pipelines?*
   A: We tune β via Bayesian optimization in the range $[0.99, 0.9999]$. When $\beta = 0$, it degrades to unweighted loss; as $\beta \to 1$, it approaches inverse frequency.



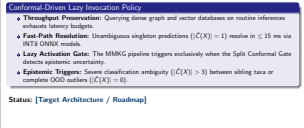
1. **Script:** Even with pre-cached embeddings, optimization dynamics matter. If you use a high learning rate on epoch 1, you can destabilize the classifier head weights. Our runner implements linear warmup to gently ramp up learning rates, followed by cosine annealing. When validation loss plateaus, our early stopping mechanism triggers and automatically restores the best-performing historical weights, guaranteeing that our deployed checkpoint is optimal.
2. **Q:** *Why not simply save weights at the very end of training?*
   A: The final epoch is frequently overfitted to the training data. Best-weight restoration guarantees that we retain the checkpoint with the highest generalization performance on holdout data.



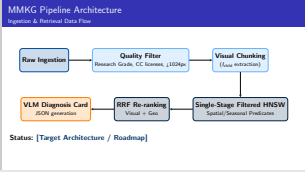
1. **Script:** Junior practitioners often tune learning rates by hand, guessing numbers in a script. That is unscientific and wastes compute. We integrate Optuna with Bayesian Tree-structured Parzen Estimators. Optuna builds a probabilistic model of which hyperparameters produce the highest validation scores. Even better, our ASHA pruner monitors intermediate epochs: if a trial is performing poorly by epoch 3, it kills it immediately and starts the next candidate, multiplying our search efficiency.
2. **Q:** *How many trials do we typically execute during an optimization run?*
   A: Because our linear probe trains in seconds, we can run 50 to 100 trials in under 10 minutes, thoroughly exploring the hyperparameter surface.



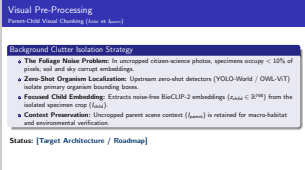
1. **Script:** A crucial question for any data scientist: how do you decide whether Model B is better than Model A? Many people look at accuracy or F1-score. But F1 requires choosing an arbitrary threshold, like 0.5. For a rare species with only three images, the optimal decision threshold might be 0.12! If you evaluate with a 0.5 threshold, F1 drops to zero. Macro PR-AUC measures the entire precision-recall curve across all possible thresholds for every single class, giving us a true, un-biased measure of classification power.
2. **Q:** *Why not use ROC-AUC instead of PR-AUC?*
   A: In extreme imbalance with 10,000 classes, each class has a massive pool of true negatives. ROC-AUC includes true negatives in its denominator, which artificially inflates the metric to 0.99 even when precision on positive samples is terrible. PR-AUC focuses strictly on positive class performance.



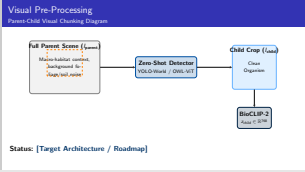
1. **Script:** Our Multimodal Knowledge Graph is powerful, but querying it takes time. In production, we do not query the graph for every single simple observation. If our conformal safety gate outputs a confident singleton prediction ($|\hat{C}(X)| = 1$), we return the INT8 ONNX prediction on the Fast Hot-Path in under 22 milliseconds. We lazy-load the MMKG only when the conformal gate signals uncertainty—either severe ambiguity ($|\hat{C}(X)| > 3$), or an empty prediction set ($|\hat{C}(X)| = 0$) indicating an anomalous, out-of-distribution specimen.
2. **Q:** *Why not query the knowledge graph in parallel on every request?*
   A: Parallel querying would saturate database connection pools and waste CPU/GPU cycles on obvious, clear-cut classifications, degrading overall cluster throughput.



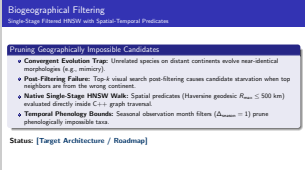
1. **Script:** This diagram maps the full data flow of the MMKG pipeline. Raw data flows through strict quality filters ensuring research-grade, open-license images over 1024 pixels. Then we extract the child organism. The isolated vector is searched in our single-stage filtered HNSW index. Retrieved candidates are fused using Reciprocal Rank Fusion, and finally passed to our local VLM to emit a structured diagnosis card.



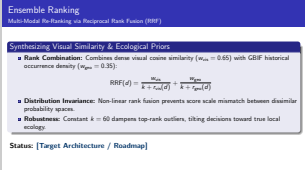
1. **Script:** In nature photography, the actual insect or wildflower often takes up less than 10 percent of the frame. The rest is dirt, grass, or human fingers. If you embed the whole image, your vector captures dirt rather than biology. We use YOLO-World or OWL-ViT to detect the specimen and isolate a clean child bounding box crop for fine-grained BioCLIP-2 embedding, while preserving the parent scene image to verify environmental habitat.
2. **Q:** *What happens if the zero-shot detector finds multiple organisms in one image?*
   A: Each detected child entity generates an independent bounding box and child embedding, allowing multi-organism observations to be analysed separately.



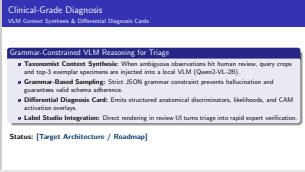
1. **Script:** This diagram visually summarises the Parent-Child Visual Chunking process. We take the full parent scene containing background noise, and pass it through a zero-shot detector like YOLO-World or OWL-ViT. This extracts the child bounding box crop, separating the organism from environmental noise, which is then fed to BioCLIP-2 to compute our clean morphological embedding vector of size 768.



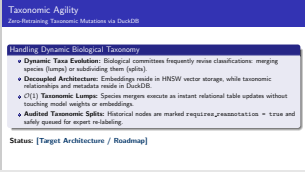
1. **Script:** Nature is full of convergent evolution: two insects on opposite sides of the globe may look identical even though they diverged 50 million years ago. If you only look at pixels, you will misclassify a European wasp as an American cicada killer. In our HNSW vector graph, we evaluate GPS coordinates and seasonal observation month directly inside the graph traversal. If an exemplar is outside a 500-kilometre radius or wrong season, it is pruned instantly before similarity scores are computed.
2. **Q:** *Why not perform vector search first and filter out GPS locations afterward?*
   A: Post-filtering suffers from candidate starvation: if your top-50 visual neighbours are all from the wrong continent, post-filtering discards all of them and leaves you with zero valid results. Native single-stage filtering guarantees high recall among geographically valid candidates.



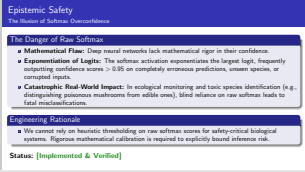
1. **Script:** Once we retrieve candidate species, how do we combine visual similarity with ecological reality? We use Reciprocal Rank Fusion. We assign a 65 percent weight to visual similarity and a 35 percent weight to historical GBIF occurrence density. Even if an exotic species looks slightly closer visually, a high local occurrence density for the native sibling species correctly tilts the ranking towards the true ecological observation.
2. **Q:** *What is the parameter k in the RRF formula?*
   A: The constant k (typically set to 60) dampens the impact of extreme outliers at top ranks, preventing a single high rank from disproportionately dominating the final ensemble score.



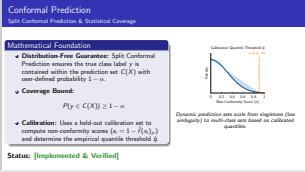
1. **Script:** When a human taxonomist is asked to review an ambiguous specimen, we don't just dump raw images on their screen. We pass the query crop and the top-3 reference exemplars into a local Vision-Language Model like Qwen2-VL-2B. Constrained by grammar-based decoding, the VLM generates a structured JSON differential diagnosis card: it details specific anatomical discriminators, shows CAM overlays, and allows the expert to confirm the species in seconds.
2. **Q:** *Why use grammar-based sampling with the Vision-Language Model?*
   A: Unconstrained VLMs hallucinate text and output unpredictable schemas. Grammar-based sampling forces the model to emit strictly valid JSON conforming to our differential diagnosis schema.



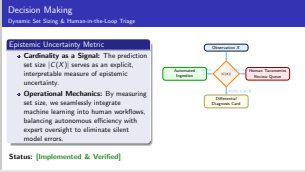
1. **Script:** In real biology, taxonomy is not static. Scientists frequently merge two species into one, or split one species into three. In a standard deep learning model, every split or merge requires retraining the entire neural network from scratch. Because our architecture decouples HNSW vector storage from DuckDB relational metadata, a taxonomic merge is an instant O(1) database update. Our vectors remain untouched with zero GPU compute cost.
2. **Q:** *What happens during a taxonomic split when one species becomes two?*
   A: Historical observation nodes for that taxon in DuckDB are flagged with `requires_reannotation = true` and purged from active traversal. Their crops are routed to Label Studio for expert re-sorting, after which they are re-registered under their new respective taxon keys.



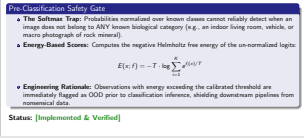
1. **Script:** Softmax outputs aren't probabilities. Networks can be 98% confident on random noise. If an app calls a deadly mushroom edible with 98% confidence, it's fatal. We must prevent uncalibrated overconfidence.
2. **Q:** *Can temperature scaling fix softmax overconfidence?*
   A: Temperature scaling adjusts the global calibration across the dataset, but it cannot provide rigorous mathematical guarantees for individual predictions or handle out-of-distribution inputs.



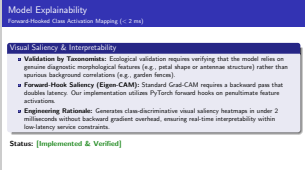
1. **Script:** TaxonVision uses Split Conformal Prediction to solve overconfidence. It guarantees (with alpha=0.05) that the prediction set contains the true species 95% of the time, without distributional assumptions.
2. **Q:** *What does a prediction set look like?*
   A: Instead of one label, it's a set. Clear image = 1 species. Ambiguous image = multiple plausible species.



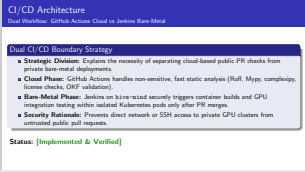
1. **Script:** In production, knowing WHEN you do not know is more valuable than making a blind guess. If our conformal prediction set has a size of 1, the model is confident and we automate the record. If the set has 2 or 3 species, we present a differential diagnosis card to the user. If the set is large or empty, our system routes it to a human review queue.
2. **Q:** *How often does an empty prediction set occur?*
   A: Empty sets occur when the model's non-conformity score for every single known class exceeds the calibrated quantile $\hat{q}$.



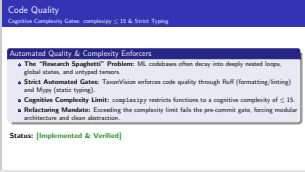
1. **Script:** What if someone points the camera at a parked car? A standard classifier might force it into a biological class like 'Black Bear'. To prevent this, we run an Energy-Based OOD detector on raw logits. Low energy indicates in-distribution, high energy flags anomalies.
2. **Q:** *Why energy vs. max softmax?*
   A: Softmax normalization destroys logit magnitude due to the denominator sum. Energy aligns with the true data density.



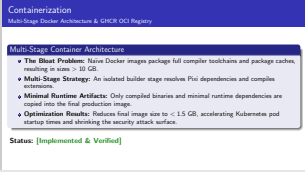
1. **Script:** Models often learn wrong shortcuts (e.g. husky vs snow). To prove our model looks at biological traits, we generate CAMs using forward hooks in ¡2ms on GPU, bypassing heavy backward passes.
2. **Q:** *Can CAM be served in real-time?*
   A: Yes. Because it takes ¡2 ms, the heatmap overlay is returned in the exact same HTTP response as the classification set.



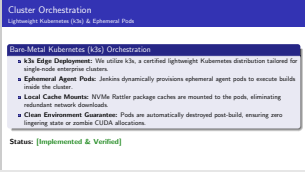
1. **Script:** Here is an enterprise DevOps pattern every student must learn: how do you connect cloud GitHub to private bare-metal servers securely? You never allow GitHub Actions to SSH directly into your private GPU cluster; that would create a massive security vulnerability. Instead, GitHub Actions runs fast cloud checks—linting, typing, and licensing—on public runners. Once code is approved, Jenkins running inside our private network pulls the signed commit and executes deployment locally.
2. **Q:** *Why not use GitHub Self-Hosted Runners directly on hive-mind?*
   A: Self-hosted runners on public or shared repos can execute untrusted code from malicious pull requests directly on your physical host machine. Ephemeral Jenkins agent pods isolate every job inside disposable container sandboxes.



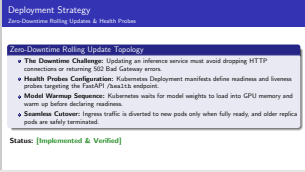
1. **Script:** Why do so many data science projects fail when moving to production? Code complexity. In Jupyter notebooks, people write 300-line functions with 6 nested loops and zero type annotations. In TaxonVision, our CI gate uses `complexipy` to measure cognitive complexity. If any function scores higher than 15, the build fails instantly. This forces engineers to write modular, single-responsibility functions that are easy to unit test and maintain.
2. **Q:** *What was an example of a refactoring forced by this complexity rule?*
   A: In `runner.py`, our embedding split preparation and index mapping functions originally scored above 20 due to combined dictionary parsing and tensor slicing. We decomposed them into dedicated helper methods, bringing complexity down to 6.



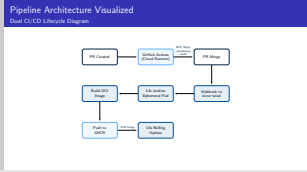
1. **Script:** In production, your deployment artifact is a container image. A novice Dockerfile copies the entire workspace and runs 'pip install', resulting in an 8-gigabyte bloated container full of build tools and temporary files. Our multi-stage Dockerfile builds dependencies in a disposable stage and copies only the compiled runtime binaries into the final image. This cuts our image down to under 1.5 gigabytes and makes Kubernetes pod deployments lightning-fast.
2. **Q:** *Why do we push images to GitHub Container Registry (GHCR)?*
   A: GHCR provides authenticated, versioned OCI storage integrated directly with our GitHub repository permissions and release tags, serving as our single source of truth for deployment artifacts.



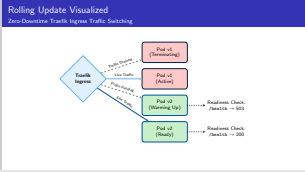
1. **Script:** On our bare-metal server, we run k3s—a lightweight, highly efficient Kubernetes distribution. When Jenkins receives a build webhook, it doesn't run the job on the host OS. Instead, it spins up an ephemeral agent container pod inside k3s, executes the build with local NVMe caching, and terminates the pod immediately. This guarantees that every single build starts in a pristine, reproducible environment with zero leftover state.
2. **Q:** *Why use k3s instead of standard k8s on a single bare-metal node?*
   A: Standard Kubernetes runs dozens of heavy control-plane components that consume gigabytes of RAM. k3s packages the entire control plane into a single binary consuming under 512 megabytes of RAM, reserving maximum system memory for our models.



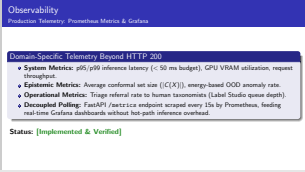
1. **Script:** Imagine updating your model in production: if you kill the old container before the new one finishes loading its 500-megabyte neural network into GPU memory, all incoming user requests fail with 500 errors. We configure Kubernetes rolling updates with readiness probes. The ingress keeps sending traffic to the old model until the new model reports that its weights are warmed up and ready. The switch happens seamlessly with zero dropped requests.
2. **Q:** *What is the difference between a liveness probe and a readiness probe?*
   A: A liveness probe checks if the process is alive; if it fails, Kubernetes restarts the pod. A readiness probe checks if the pod is ready to accept HTTP traffic; if it fails, traffic is diverted without killing the container.



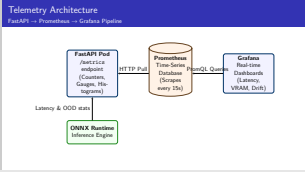
1. **Script:** This diagram visualizes our asymmetric pipeline. Notice the top half is the public cloud—GitHub Actions running lightweight static analysis. The bottom half is our secure bare-metal enclave. The only bridge between them is the webhook triggered on PR merge, ensuring that unvetted code never executes on our primary GPU hardware. Jenkins then builds the container, pushes it back to the cloud registry, and manages the local Kubernetes deployment.
2. **Q:** *Why does Jenkins build the image instead of GitHub Actions?*
   A: Building a production ML container often requires compiling custom CUDA kernels or resolving large dependencies. Doing this on GitHub's free runners is slow and resource-constrained. Our bare-metal machine leverages local NVMe caches and powerful CPUs for rapid builds.



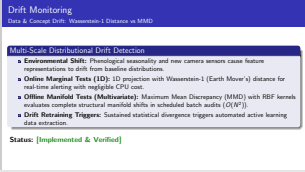
1. **Script:** This diagram shows the transition state during a deployment. Traefik is the ingress controller mapping external requests. When we deploy Version 2, the new pod initially fails its readiness probe while the model weights load into the GPU. Traefik does not send any traffic to it. Meanwhile, Version 1 continues serving users. Only when Version 2 returns a 200 OK on its health endpoint does Traefik route traffic to it, and subsequently, Kubernetes begins draining and terminating the Version 1 pods.
2. **Q:** *What happens if the new model has a bug and never passes the readiness probe?*
   A: The deployment halts. Version 2 remains in a pending state, and Traefik continues routing 100% of traffic to Version 1 indefinitely. Users experience no disruption, and the engineering team is alerted to the failed deployment.



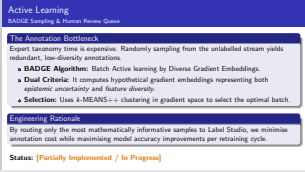
1. **Script:** Observability in MLOps is fundamentally different from traditional DevOps. In web development, you monitor CPU and HTTP status codes. In MLOps, an API can return HTTP 200 OK all day while predicting complete nonsense! We monitor domain-specific metrics: average conformal prediction set size, percentage of samples routed to human review, and OOD energy distributions. If average set size jumps from 1.2 to 4.5, we know immediately that the data distribution has shifted.
2. **Q:** *How does Prometheus scrape metrics from our service?*
   A: FastAPI instruments latency and counters using the `prometheus_client` library, exposing a `/metrics` endpoint that the Prometheus server polls every 15 seconds.
3. **Q:** *Why not push metrics directly to a database?*
   A: The pull-based model of Prometheus ensures that if the monitoring system goes down, the inference service is unaffected. It drops the scraping request rather than blocking the main thread on a database timeout.



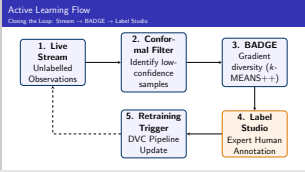
1. **Script:** This diagram illustrates our telemetry flow. The core inference engine passes latency and out-of-distribution statistics up to the FastAPI layer, which aggregates them in memory using the Prometheus client. The Prometheus server periodically pulls these metrics, and Grafana queries Prometheus to build the dashboards you see in the control room.
2. **Q:** *Why does Prometheus pull rather than the service pushing metrics?*
   A: Pull-based monitoring prevents the application from hanging if the metrics server goes down or becomes slow, ensuring zero impact on the critical inference path.



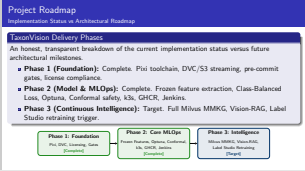
1. **Script:** Machine learning models degrade over time because the real world changes—we call this data drift. If autumn arrives, leaves turn yellow and lighting shifts, but our training set was collected in summer. We use the Wasserstein-1 Earth Mover's Distance to monitor the distribution of incoming feature embeddings against our baseline training distribution. When the statistical distance crosses our alert threshold, the system flags the drift and prepares an active learning cycle.
2. **Q:** *Why not run multivariate MMD on every single incoming inference request?*
   A: Computing full-manifold MMD across high-dimensional embeddings has quadratic complexity $O(N^2)$ in sample size. We perform 1D Wasserstein distance on projected features in real time and reserve full MMD for periodic offline batch audits.



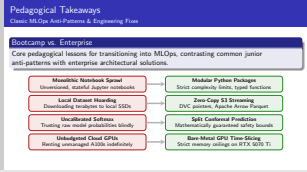
1. **Script:** Annotation by expert taxonomists is expensive and slow. If you send them 10,000 photos of the same common dandelion, you waste their time. BADGE solves this by analyzing the gradients that each unlabelled image would induce in the model. It uses k-MEANS++ clustering in gradient space to pick a batch of images that are both highly uncertain AND visually diverse. This maximizes the model's accuracy improvement per dollar spent on human labeling.
2. **Q:** *What is the current status of the active learning pipeline?*
   A: The BADGE mathematical sampling algorithm and Label Studio export formats are fully implemented in `src/taxon_vision/active_learning/badge.py`. The automated webhook that triggers unattended model retraining upon annotation completion is our Phase 3 target milestone.



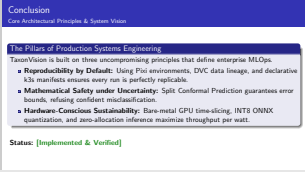
1. **Script:** This flow shows how active learning operates in production. Unlabelled data flows in from our web streams. The split conformal prediction layer filters out cases where the model is highly confident. The remaining low-confidence samples are processed by the BADGE algorithm, which clusters their gradients to select a small, diverse batch. This batch is sent to Label Studio for expert taxonomists to review. Once reviewed, it triggers a DVC pipeline update, retraining the model and updating the live stream.
2. **Q:** *How does the Conformal Filter help BADGE?*
   A: By filtering out high-confidence samples first, we significantly reduce the size of the dataset BADGE needs to process, making the $O(N \log N)$ k-MEANS++ clustering computationally feasible.



1. **Script:** Let's review our progress transparently. Phase 1 and Phase 2 are complete and battle-tested in our repository today: our reproducible Pixi environment, S3 zero-copy streaming, Class-Balanced Loss runner, conformal safety bounds, Jenkins container builds, and Kubernetes manifests are live. Phase 3—connecting the Milvus vector database into our active learning webhook loop—is our ongoing target architecture roadmap.
2. **Q:** *How do we ensure that new feature development does not regress existing Phase 1 and 2 capabilities?*
   A: Every commit is protected by our pre-commit gates, automated test fixtures in `tests/`, and CI pipeline validation including license and complexity audits.



1. **Script:** For all of you in this course, if you take away four key lessons from this project, make it these four: First, stop writing monolithic notebooks—use modular packages with strict complexity limits. Second, don't download multi-terabyte datasets locally—stream them and cache embeddings. Third, never trust raw softmax probabilities—use conformal prediction sets for safety. And fourth, don't waste thousands of dollars on unmanaged cloud GPUs—budget your VRAM and use time-slicing.
2. **Q:** *What is the fastest way for a junior data scientist to adopt these best practices?*
   A: Start by enforcing automated pre-commit hooks with Ruff and Mypy, tracking datasets with DVC, and writing unit tests for your data transformation functions before touching model code.



1. **Script:** To conclude: MLOps is not just about using trendy tools like Kubernetes or Docker. It is a systems engineering mindset. It means building models that are reproducible down to the exact commit, mathematically honest about what they don't know, and engineered to run reliably on physical hardware within budget. TaxonVision proves that with the right architecture, we can tackle the most complex biodiversity challenges on Earth with mathematical confidence and operational excellence. Thank you!
2. **Q:** *Where can students inspect the source code and documentation?*
   A: The entire repository, including Zensical documentation, LaTeX strategy reports, and CI/CD pipelines, is maintained under strict version control in the taxon-vision-mlops repository.